Always

- An always block can include a sensitivity list if any of these signals change then the always block is executed

```
always@ (a or b or c)
statement;
```

```
always@ (a, b, c)
statement;
```

- In Verilog-2001 we can use the@(*) construct which creates a sensitivity list for all signals read in the always block

```
always@ (*)
statement;
```

Blocking Assignment

- Blocking procedural assignments:

<value> = *<timing_control>* *<expression>*

  - Similar to C and Pascal assignments
  - must be executed before the execution of the statements that follow them in a sequential block
    - Executed in the order they appear
  - When RHS has more bits than LHS:
    - Truncate the MSB
  - When LHS has more bits than RHS:
    - Zeros are filled in MSB
  - does not prevent the execution of statements that follow them in a parallel block

Non-Blocking Assignment

- Non-blocking procedural assignments:

<lvalue> **<=** *<timing_control>* *<expression>*

  - allow to schedule assignments without blocking the procedural flow
  - used whenever you want to make several register assignments within the same time step without regard to order or dependence upon each other
  - they are executed in two steps:
    - (a) the simulator evaluates the right-hand side,
    - (b) the assignment occurs at the end of the time step indicated by the *<timing_control>*

Blocking vs. Nonblocking Assignments

- Blocking assignment: evaluation and assignment are immediate

```
always @ (a or b or c)
begin
  x = a | b;          1. Evaluate a | b, assign result to x
  y = a ^ b ^ c;      2. Evaluate a^b^c, assign result to y
  z = b & ~c;         3. Evaluate b&(~c), assign result to z
end
```

- Nonblocking assignment: all assignments deferred until all right-hand sides have been evaluated (end of simulation timestep)

```
always @ (a or b or c)
begin
  x <= a | b;         1. Evaluate a | b but defer assignment of x
  y <= a ^ b ^ c;     2. Evaluate a^b^c  but defer assignment of y
  z <= b & ~c;        3. Evaluate b&(~c) but defer assignment of z
end                   4. Assign x, y, and z with their new values
```

Dataflow: Full Adder with Verilog operators

```
1  module FullAdder(a,b,cin,sum,cout);
2      input a,b,cin;
3      output sum, cout;
4      assign sum <= a ^ b ^ cin;
5      assign cout <= (a & b) | (a & cin) | (b & cin);
6      end
7  endmodule
```

```
1  module FullAdder(a,b,cin,sum,cout);
2      input a,b,cin;
3      output sum, cout;
4      reg sum, cout; // registers retain value
5      always @ (a or b or cin) begin
6          sum = a ^ b ^ cin;
7          cout = (a & b) | (a & cin) | (b & cin);
8      end
9  endmodule
```




Behavioral Level Modeling

- Module can be implemented in terms of the desired design algorithm without concern for the hardware implementation details.
- An ***always*** block is a behavioral block which contains a list of expressions which are (usually) evaluated sequentially
- The code in an always block can be very abstract very similar to C programming
- Always blocks can contain ***case*** statements, ***for*** loops, ***while*** loops, even ***functions*** – they enable high-level behavioral modeling



Behavioral : 4-input Multiplexer with an if/else statement

```
1  module mux4( input a, b, c, d,
2               input [1:0] sel,
3               output out );
4               reg out;
5  always @( a or b or c or d or sel ) begin
6      if ( sel == 0 )
7          out = a;
8      else if ( sel == 1 )
9          out = b;
10     else if ( sel == 2 )
11         out = c;
12     else if ( sel == 3 )
13         out = d;
14 end
15 endmodule
```

very similar to C programming

Behavioral : 4-input Multiplexer with case statement

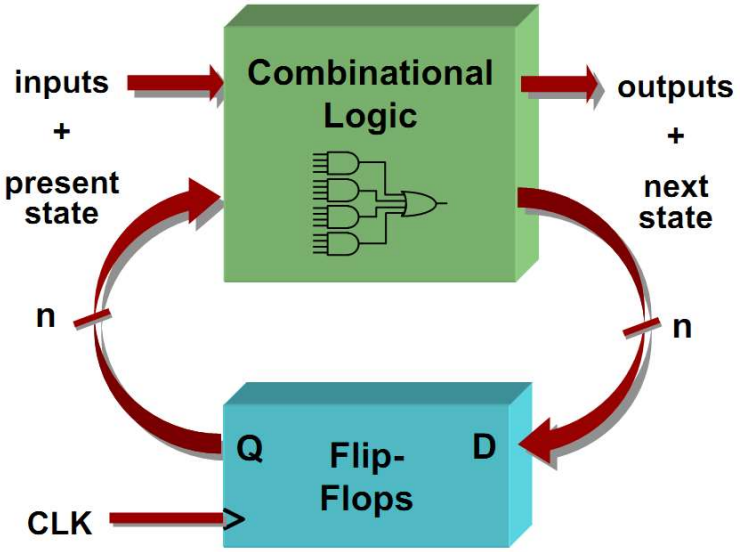
- case and if may be used interchangeably to implement conditional execution within always blocks

```
1  module mux4( input a, b, c, d,
2               input [1:0] sel,
3               output out );
4               reg out;
5  always @( a or b or c or d or sel ) begin
6      if ( sel == 0 )
7          out = a;
8      else if ( sel == 1 )
9          out = b;
10     else if ( sel == 2 )
11         out = c;
12     else if ( sel == 3 )
13         out = d;
14 end
15 endmodule
```

```
1  module mux4( input a, b, c, d,
2               input [1:0] sel,
3               output out );
4               reg out;
5  always @( * ) begin
6      case (sel)
7          0: out = a;
8          1: out = b;
           2: out = c;
10         3: out = d;
11     endcase
12 end
13 endmodule
```

case is easier to read than a long string of if...else statements

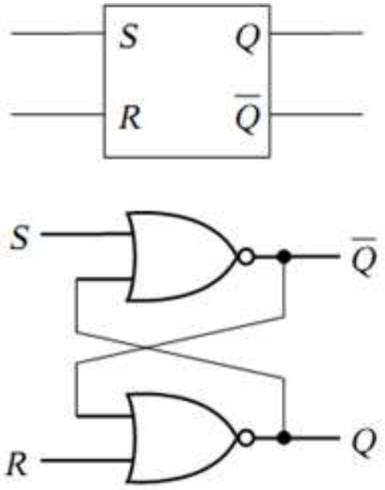
Sequential Logic in Verilog



- Sequential circuits have memory
  - Flip-Flops, Latches,
    Finite State Machines

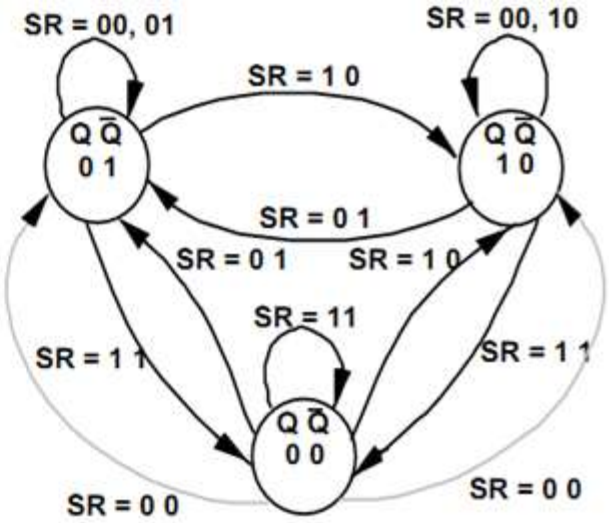
- The current state is "held" in memory and the next state is computed based on the current state and the current inputs
- Sequential Logic is triggered by a 'CLOCK' event
  - Latches are sensitive to level of the signal
  - Flip-flops are sensitive to the transitions of clock

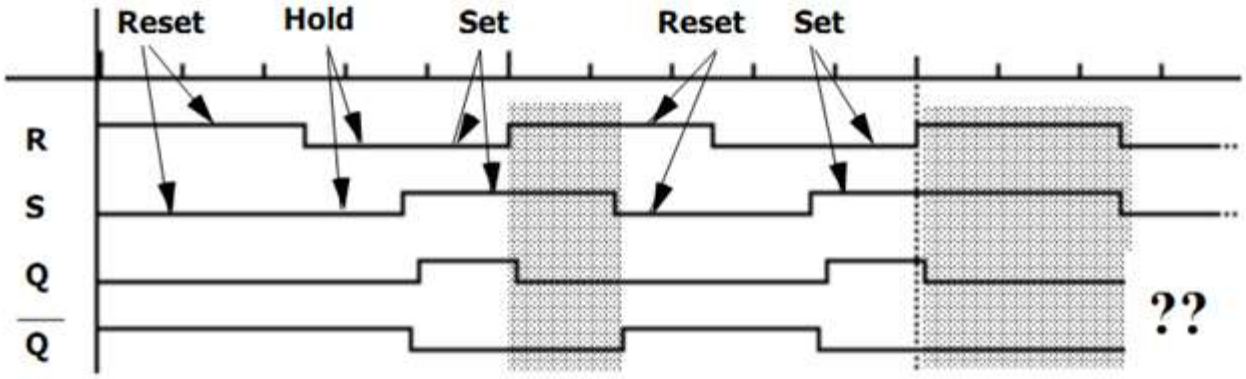
NOR-based Set-Reset (SR) Latch



S	R	Q	$\overline{Q}$
0	0	Q	$\overline{Q}$
1	0	1	0
0	1	0	1
1	1	0	0

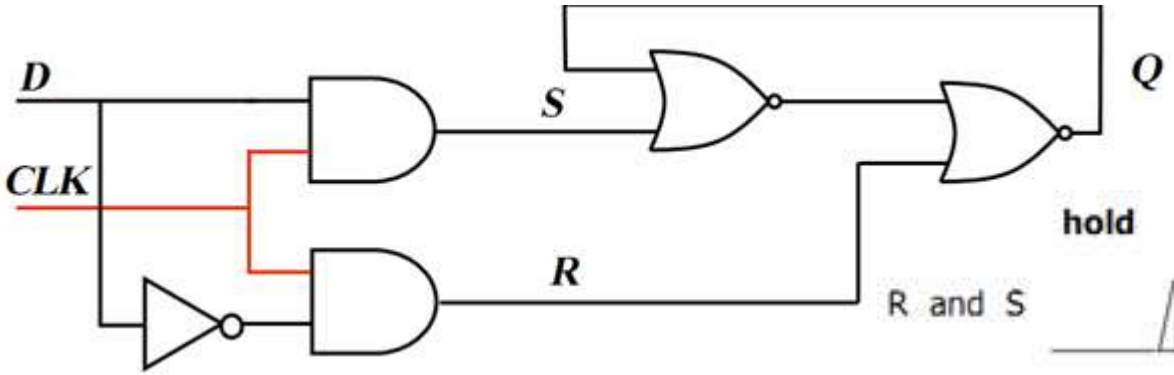
Forbidden State

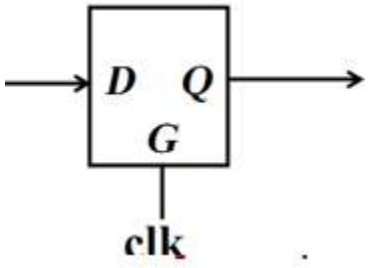


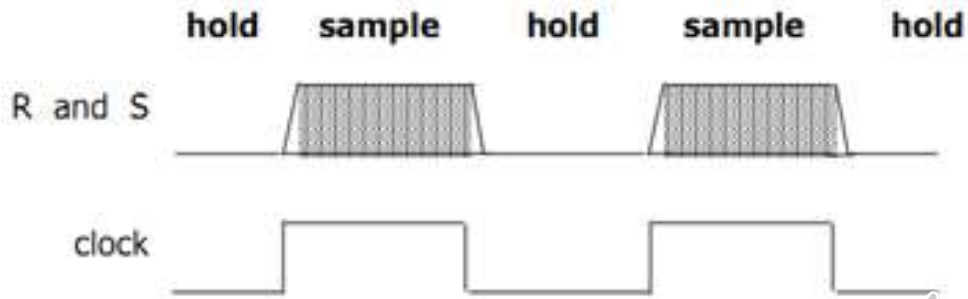


Making a Clocked Memory Element: Positive D-Latch

- A Positive D-Latch:
  - Passes input D to output Q when CLK is high
  - holds state when clock is low (i.e., ignores input D)
- This following Latch is level-sensitive
  - invert clock for a negative latch





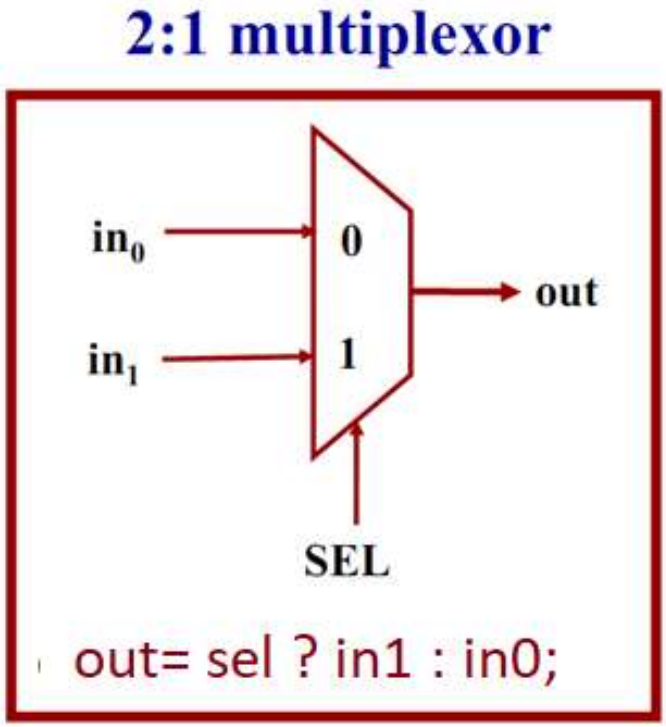


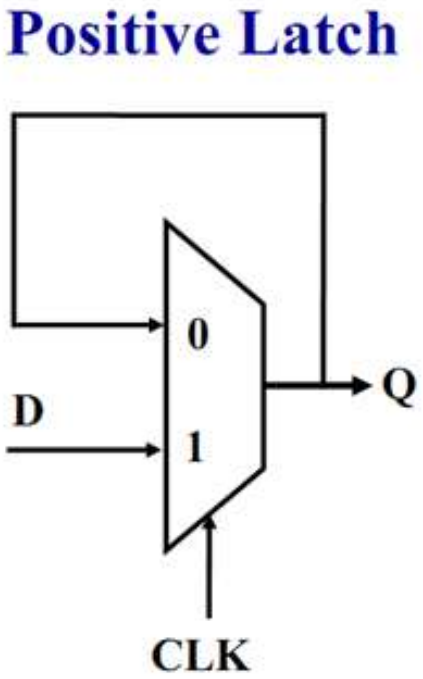
D Latch

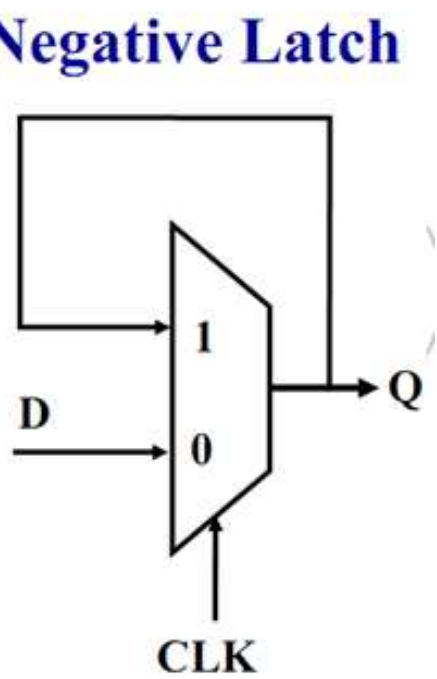
```
1  module latch
2       (input clk,
3       input [3:0] d,
4       output reg[3:0] q);
5  always @ (clk, d)
6       if(clk) q <= d;
7  endmodule
```

latch is transparent when clock is 1

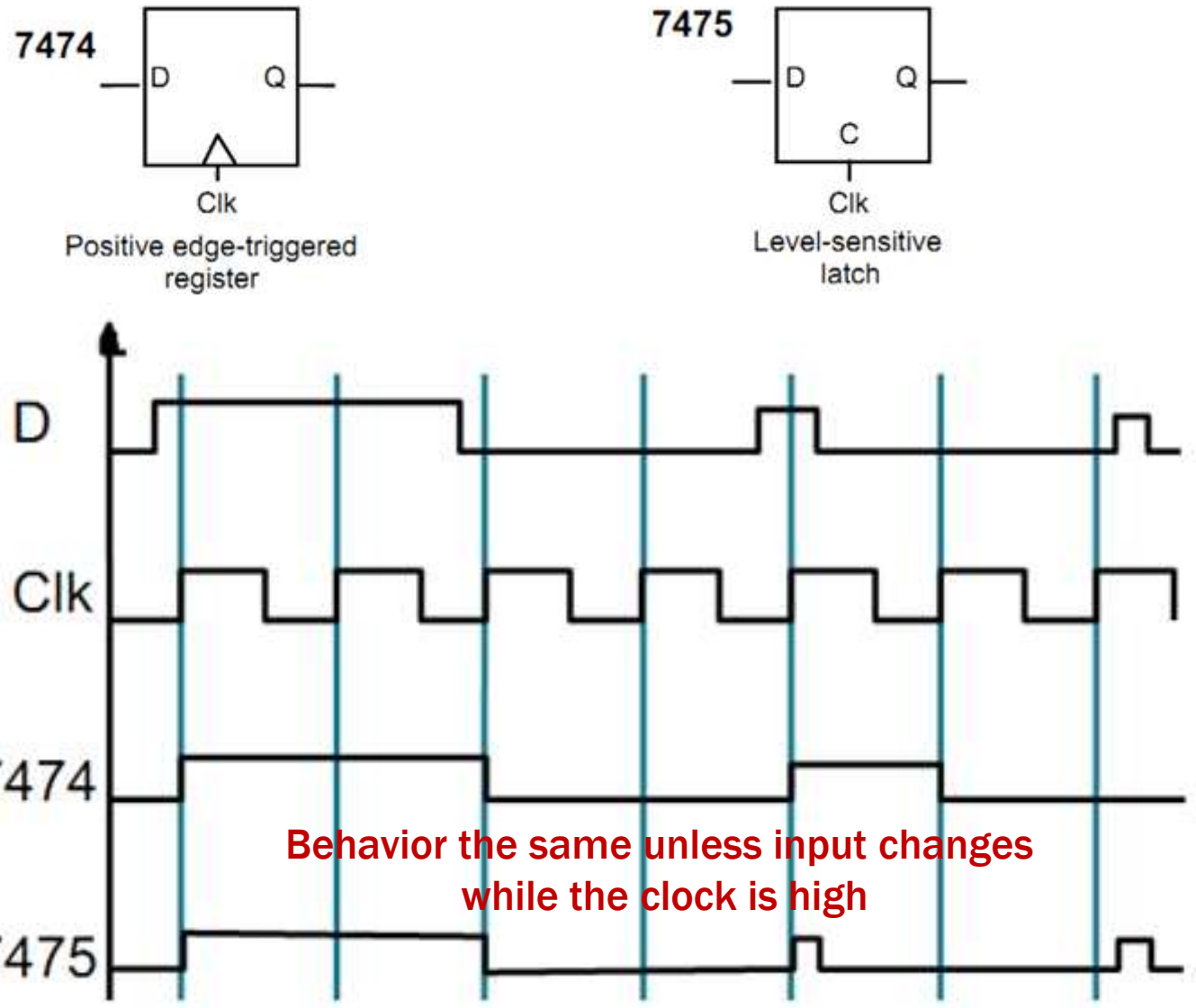
Multiplexor Based Positive & Negative Latch







Latches vs. Edge-Triggered Register



D Flip-Flop

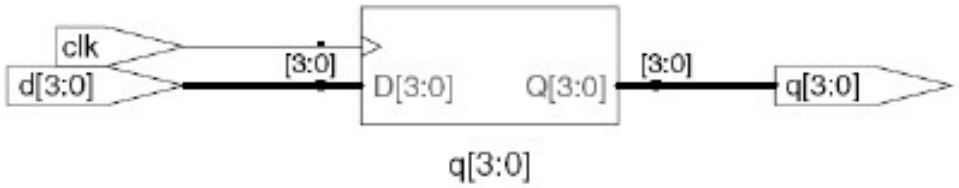
The posedge defines a rising edge (transition from 0 to 1) This process will trigger only if the clk signal rises

Once the clk signal rises: the value of d will be copied to q

```
1  module FF_1bit
2      (input clk,
3      input d,
4      output reg q
5      );
6  always@ (posedge clk)
7      q <= d; // pronounced "q gets d"
8  endmodule
9
```

4-bit flip flop can be simply modeled by defining d and q as 4-bit vectors

```
1  module FF_4bit
2      (input clk,
3      input[3:0] d,
4      output reg[3:0] q
5      );
6  always@ (posedge clk)
7      q <= d; // pronounced "q gets d"
8  endmodule
```



D Flip-Flop with Asynchronous Reset

```
1   module FF_4bit_ar
2       (input clk,
3        input reset,
4        input [3:0] d,
5        output reg[3:0] q
6        );
7   always @ (posedge clk, negedge reset)
8   begin
9        if (reset == 0)
10           q <= 0; // when negedge reset
11       else
12           q <= d; // when posedge clk
13   end
14  endmodule
```

two events can trigger this process:
A rising edge on clk
A *falling edge* on reset

First reset is checked, if reset is 0, q is set to 0.

This is an 'asynchronous' reset as the reset does not care what happens with the clock

D Flip-Flop with Synchronous Reset

```
1   module FF_4bit_sr
2       (input clk,
3       input reset,
4       input [3:0] d,
5       output reg[3:0] q
6       );
7   always @ (posedge clk)
8   begin
9       if (reset == 0)
10          q <= 0; // when clk and ~reset
11      else
12          q <= d; // when clk and reset
13  end
14  endmodule
```

The process is only sensitive to posedge clock

Reset *only happens* when the *clock rises*. This is a 'synchronous' reset

D Flip-Flop with Enable and Reset

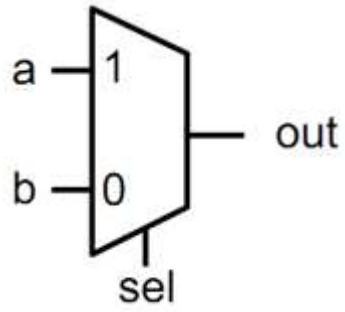
```
1   module FF_4bit_ar
2       (input clk,
3       input reset,
4       input [3:0] d,
5       input en,
6       output reg[3:0] q
7       );
8   always @ (posedge clk, negedge reset)
9   begin
10      if (reset == 0)
11          q <= 0; // when negedge reset
12      else if (en)
13          q <= d; // when en AND posedge clk
14   end
15  endmodule
```

only when “clk is rising *AND* en is 1” data is stored

Edge-triggered circuits are described using a sequential always block

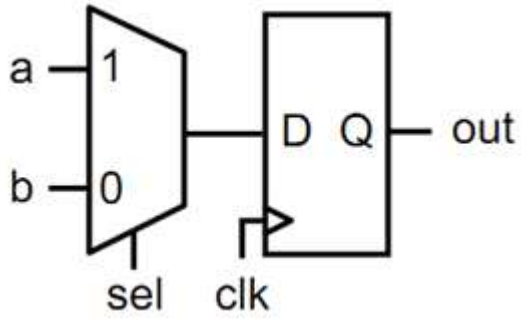
Combinational

```
1  module comb( input a, b,
2               input sel,
3               output out );
4               reg out;
5   always @( a or b or sel )
6  begin
7       if (sel) out = a;
8       else out = b;
9  end
10  endmodule
```



Sequential

```
1  module seq( input a, b,
2              input sel,
3              output out );
4              reg out;
5   always @(posedge clock)
6  begin
7       if (sel) out <= a;
8       else out <= b;
9  end
10  endmodule
```



How to use non-blocking and blocking

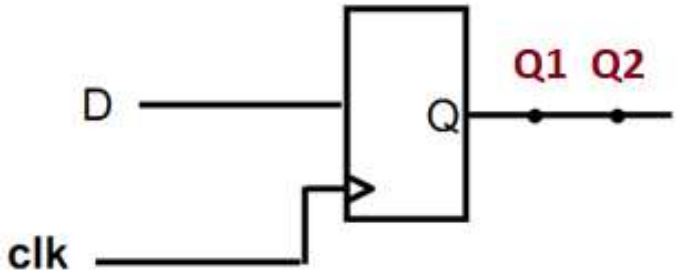
```
1   module myAdder
2       (input clock,
3        input           [7:0]  in1,
4        input           [7:0]  in2,
5        output reg [7:0]  f1, f2, f3, f4, f5
6       );
7       // f1 will be a flipflop
8       always @(posedge clock) begin
9           f1 = in1 + in2;
10      end
11
12      // f2 will be a flipflop
13      always @(posedge clock) begin
14          f2 <= in1 + in2;
15      end
```

"Blocking"
Will be synthesized to FF
(not recommended)

"Non-blocking"
Will be synthesized to FF

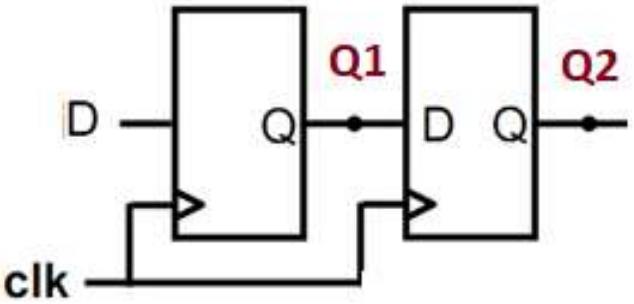
How to use non-blocking and blocking

```
1  module DFF-blocking
2      (input D, Clock,
3      output Q1, Q2
4      );
5      reg Q1, Q2;
6  always @(posedge Clock)
7  begin
8      // blocking assignment
9      Q1 = D;
10     Q2 = Q1;
11 end
12 endmodule
```



at each clock rising edge:
Q1 = D after that Q2=Q1=D
Therefore Q2=D

```
1  module DFF-nonblocking
2      (input D, Clock,
3      output Q1, Q2
4      );
5      reg Q1, Q2;
6  always @(posedge Clock)
7  begin
8      // non-blocking assignment
9      Q1 <= D;
10     Q2 <= Q1;
11 end
12 endmodule
```



at each clock rising edge:
Q1 and Q2 simultaneously receive
the old values of D and Q1

How to use non-blocking and blocking

```
1   // f3 will be a flipflop
2   // c1 will be a flipflop
3      reg [7:0] c1;
4      always @(posedge clock) begin
5         c1 <= in1 + in2;
6         f3  <= c1 + in1;
7      end
8
9      // f4 will be a flipflop
10     // c2 is used only within the always block
11     // and so is treated as a tmp variable
12     // and won't be inferred as a flipflop
13     reg [7:0] c2;
14     always @(posedge clock) begin
15        c2 = in1 + in2;
16        f4  = c2 + in1;
17     end
```

"Non-blocking"
Will be synthesized to FF

"Blocking"
Simulation and
Synthesis Differ
(not recommended)

Race Condition

```
1   //Example 1:
2   // There is a race condition
3   always @(posedge clock)
4       b = a;
5   always @(posedge clock)
6       a = b;
7
8   //Example 2:
9   // There is no race
10  always @(posedge clock)
11      b <= a;
12  always @(posedge clock)
13      a <= b;
14
```

How values of ***a*** or ***b*** should be evaluated. Which one is first???

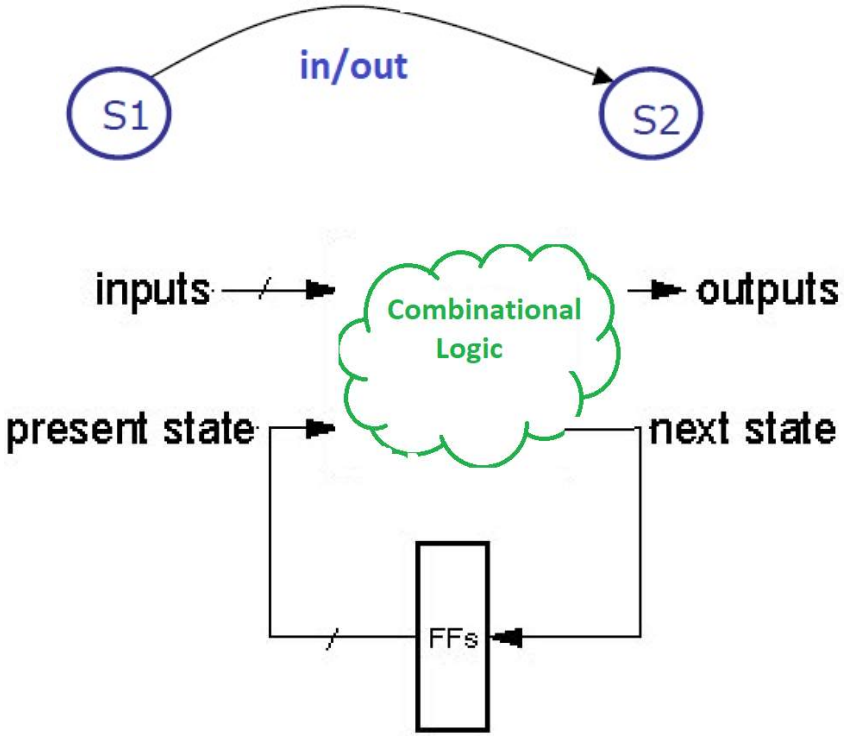
Registers a and b are swapped.



Finite State Machines - 1

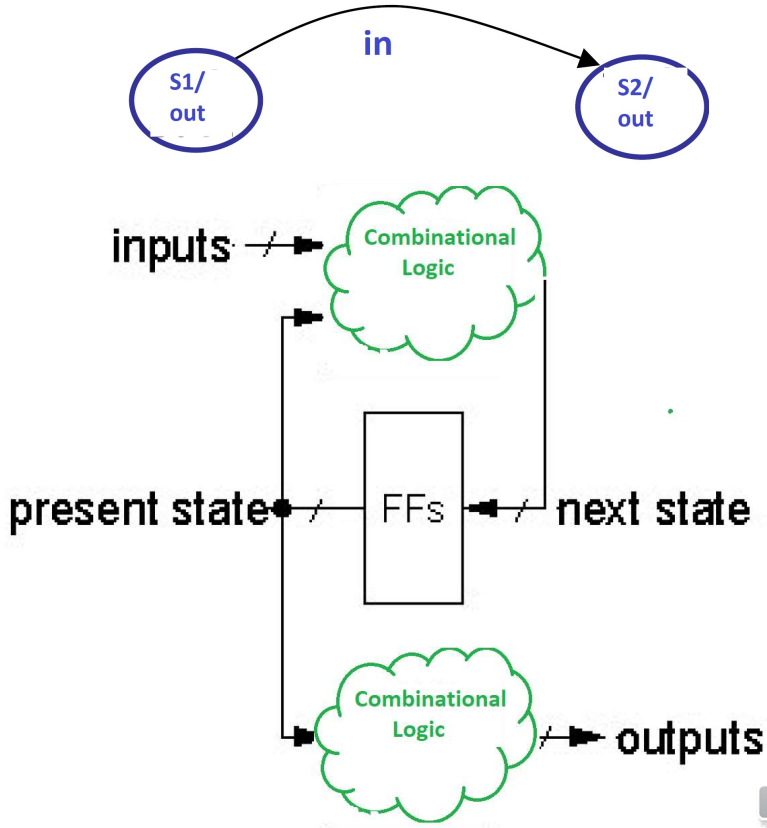
Mealy FSM

Output based on present state and input



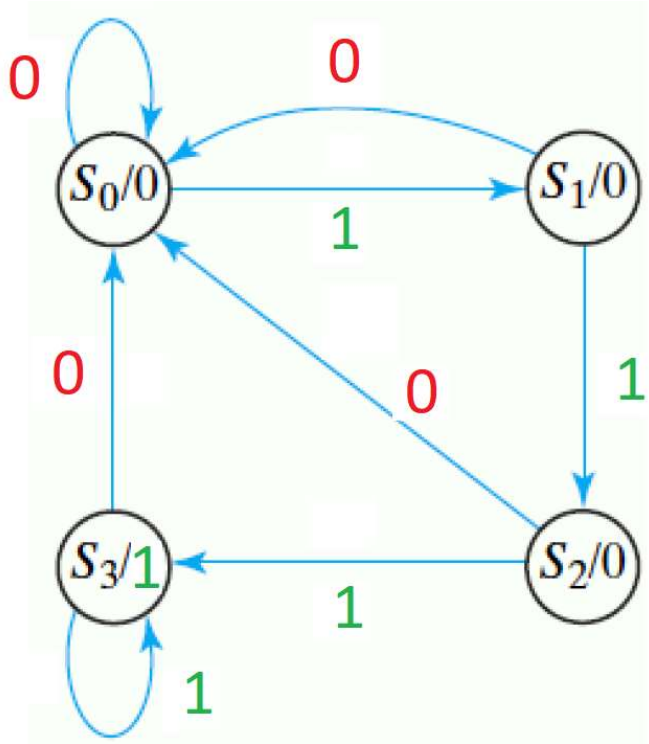
Moore FSM

Output only based on present state



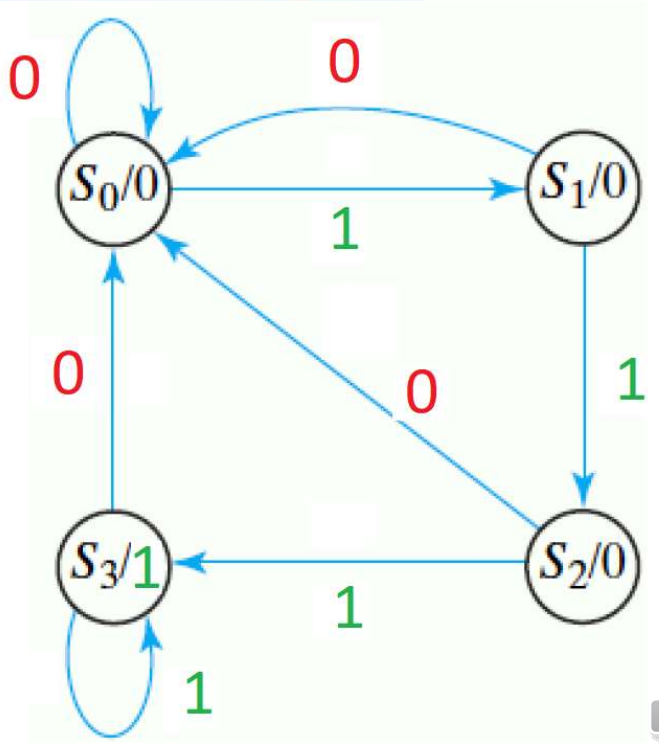
Example: Sequence Detector

- Circuit specification:
  - Design a circuit that outputs a 1 when three consecutive 1's have been received as input and 0 otherwise.
- FSM type
  - Moore or Mealy FSM?
    - Both possible
- State diagram:
  - State S0: zero 1s detected
  - State S1: one 1 detected
  - State S2: two 1s detected
  - State **S3: three 1s detected**



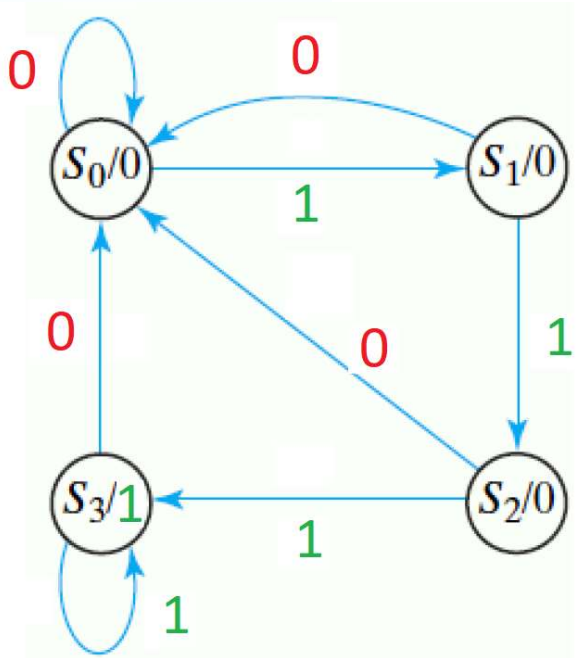
Sequence Detector: Verilog (Moore FSM 1)

```
1  module seq_detect_moore(x,clk, y);
2  // Moore machine for a three-1s sequence detection
3      input x, clk;
4      output y;
5      reg [1:0] state;
6      parameter S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
7  // Define the sequential block
8      always @(posedge clk)
9          case (state)
10             S0: if (x) state <= S1;
11             else state <= S0;
12             S1: if (x) state <= S2;
13             else state <= S0;
14             S2: if (x) state <= S3;
15             else state <= S0;
16             S3: if (x) state <= S3;
17             else state <= S0;
18         endcase
19 // Define output during S3
20     assign y = (state == S3);
21 endmodule
```



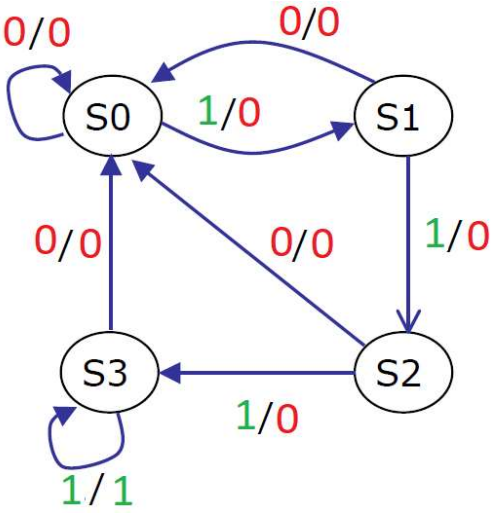
Sequence Detector: Verilog (Moore FSM 2)

```
1  module seq_detect_moore(x,clk, y);
2  // Moore machine for a three-1s sequence detection
3      input x, clk;
4      output y;
5      reg [1:0] nstate, pstate; //next state and present state
6      parameter S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
7  // Next state combinational logic
8  // Use blocking assignments "="
9      always @(x or pstate)
10         case (pstate)
11             S0: if (x) nstate = S1;
12             else nstate = S0;
13             S1: if (x) nstate = S2;
14             else nstate = S0;
15             S2: if (x) nstate = S3;
16             else nstate = S0;
17             S3: if (x) nstate = S3;
18             else nstate = S0;
19         endcase
20 // Define output
21     assign y = (pstate == S3);
22 // State transition
23 // Sequential logic, use nonblocking assignments "<="
24 always @(posedge clk)
25     pstate <= nstate;
26 endmodule
```



Sequence Detector: Verilog (Mealy FSM)

```
1  module seq3_detect_mealy(x,clk, y);
2  // Mealy machine for a three-1s sequence detection
3      input x, clk;
4      output y;
5      reg y;
6      reg [1:0] pstate, nstate; //present and next states
7      parameter S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
8      // Next state and output combinational logic
9      // Use blocking assignments "="
10     always @(x or pstate)
11         case (pstate)
12             S0: if (x) begin nstate = S1; y = 0; end
13             else begin nstate = S0; y = 0; end
14             S1: if (x) begin nstate = S2; y = 0; end
15             else begin nstate = S0; y = 0; end
16             S2: if (x) begin nstate = S3; y = 0; end
17             else begin nstate = S0; y = 0; end
18             S3: if (x) begin nstate = S3; y = 1; end
19             else begin nstate = S0; y = 0; end
20         endcase
21     // Sequential logic, use nonblocking assignments "<="
22     always @(posedge clk)
23         pstate <= nstate;
24 endmodule
```

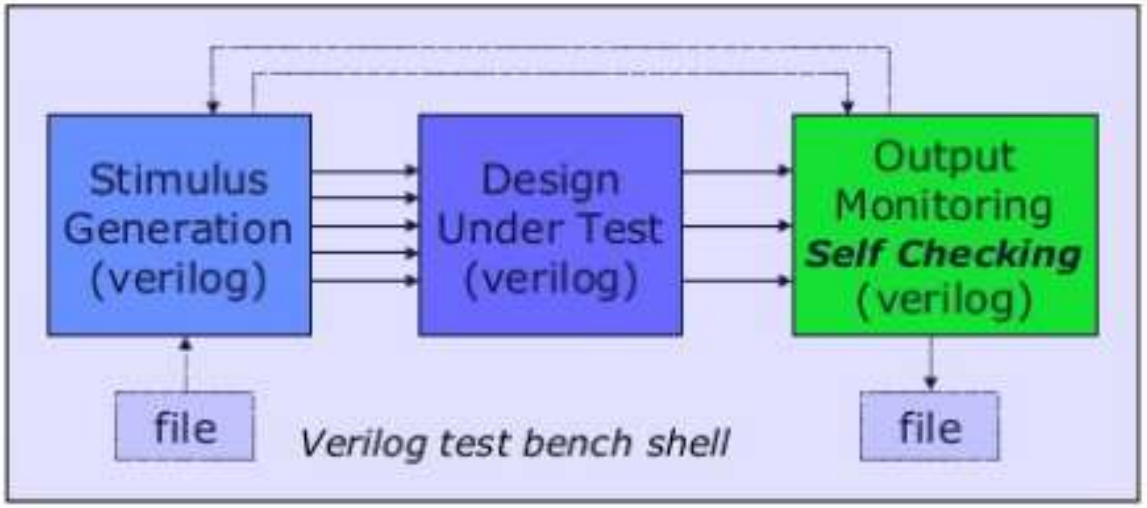


Simulating/Validating HDL

- The sad truth…
  - 10% design, 90% validation
  - If you do it right, you will spend 9x more time testing/validating a design than designing it

Testbenchs are written in verilog as well.

Testbench verilog is not describing hardware and can be thought of as more of a program.



Testbench Example

```
1   module testbench;
2       reg aa,bb,ccin;
3       wire ss,ccout;
4
5       adder add1(.a(aa),.b(bb),.cin(ccin),
6           .s(ss),.cout(ccout));
7       //aa, bb, ccin can be reg or net
8       //ss, and ccout have to be net
9
10      initial begin
11              aa=1'b0; bb=1'b1; ccin=1'b0;
12          #2  aa=1'b1; bb=1'b1; ccin=1'b1;
13          #2  aa=1'b0; bb=1'b0; ccin=1'b0;
14          #2  aa=1'b1; bb=1'b1; ccin=1'b0;
15          #2  $stop;
16      end
17
18      initial
19          $monitor($time,"aa=%b, bb=%b, ccin=%b,
20              ss=%b, ccout=%b", aa,bb,ccin,ss,ccout);
21  endmodule
```

```
1   module adder(a,b,cin,s,cout);
2       output s, cout;
3       input a,b,cin;
4       reg s, cout;
5       always @(a or b or cin)
6           begin
7           s=a^b^cin;
8           cout=a&b | a&cin | b&cin;
9       end
10  endmodule
```




Verilog tips

- **Do not** write C-code
  - Think hardware, not algorithms
    - Verilog is **inherently parallel**
    - Compilers don't map algorithms to circuits well
- **Do** describe hardware circuits
  - First draw a dataflow diagram
  - Then start coding
- Assign any signal or variable from only one source
- Use blocking assignments for combinational circuits
- Use non-blocking assignments for sequential circuits
- If one signal in the sensitivity list uses posedge/negedge, then all signals must

